Learn to create Python GUIs using Tkinter

February 7, 2026

1 Introduction

In this course, you will learn how to create Graphical User Interfaces (GUIs) using the Tkinter library in Python.
Tkinter is a standard Python library that provides a simple way to create GUI applications. It is cross-platform, meaning it can be used on Windows, macOS, and Linux.
It's simple and great for beginners.

2 What you will learn

- How to create a basic window with Tkinter
- How to add labels, buttons, and other widgets to a window
- How to handle user input and events
- How to create more complex layouts and designs
- How to use images and other media in your GUIs

3 Prerequisites

- Basic knowledge of Python programming
- Basic knowledge of object-oriented programming

4 Getting started

Tkinter comes pre-installed with Python. You can import it using:

```
import tkinter as tk
```

Once you have installed Tkinter, you can start creating your first GUI application.

5 Creating a basic window

To create a basic window with Tkinter, you will need to import the Tkinter library and create a window object. Here is an example:

```
import tkinter as tk
window = tk.Tk()
window.mainloop()
```

This code creates a window object and starts the Tkinter event loop. The event loop is what allows your GUI to respond to user input and events.

6 Adding widgets to a window

Once you have created a window, you can add widgets to it. Widgets are the building blocks of a GUI, such as buttons, labels, and text boxes. Here is an example of how to add a label to a window:

```
import tkinter as tk
window = tk.Tk()
label = tk.Label(window, text='Hello, world!')
label.pack()
window.mainloop()
```

This code creates a label widget and adds it to the window using the `pack()` method. The `pack()` method is one of the ways to add widgets to a window. There are other methods, such as `grid()` and `place()`, that you can use to add widgets to a window.

```
import tkinter as tk

window = tk.Tk()
label = tk.Label(window, text="Hello")
button = tk.Button(window, text="Click")
entry = tk.Entry(window)

label.pack()
button.pack()
entry.pack()
window.mainloop()
```

7 Handling user input and events

To handle user input and events, you will need to use event handlers. Event handlers are functions that are called when a specific event occurs, such as when a button is clicked or a key is pressed. Here is an example of how to create an event handler for a button:

```
import tkinter as tk
def button_clicked():
    entry.delete(0, tk.END)
    entry.insert(tk.END, "Button clicked!")

window = tk.Tk()

button = tk.Button(window, text='Click me!', command=button_clicked)
button.pack()

entry = tk.Entry(window)
entry.pack()
```

```
window.mainloop()
```

This code creates a button widget and adds it to the window using the `pack()` method. It creates also an Entry widget and adds it to the window using the `pack()` method. The `command` parameter of the `Button` widget is set to the `button_clicked()` function, which is called when the button is clicked. It deletes the content of the entry, then inserts "Button clicked!" into the entry.

8 Conclusion

In this course, you have learned how to create basic GUI applications using Tkinter in Python. You have learned how to create a window, add widgets to a window, handle user input and events. Tkinter is a powerful library that can be used to create a wide range of GUI applications, from simple tools to complex desktop applications.